

Lecture 3: Python Collections

Beginner Notes | Python for GenAI

1. Lists

A list is an ordered collection of values. Lists can store multiple items, allow duplicate values, and can be changed after they are created.

```
languages = ["Python", "Java", "C++"]
print(languages)
print(languages[0])
languages.append("JavaScript")
print(languages)
```

Lists use square brackets `[]`. Python uses zero-based indexing, so the first item is at position 0. A list can contain different data types, although keeping related values together usually makes code easier to understand.

Common list operations

Operation	Meaning	Example
<code>append()</code>	Add one item at the end	<code>languages.append("Go")</code>
<code>insert()</code>	Add an item at a position	<code>languages.insert(1, "Rust")</code>
<code>remove()</code>	Remove the first matching value	<code>languages.remove("Java")</code>
<code>pop()</code>	Remove and return an item	<code>languages.pop()</code>
<code>sort()</code>	Sort the list	<code>numbers.sort()</code>
<code>reverse()</code>	Reverse the list	<code>numbers.reverse()</code>
<code>len()</code>	Get number of items	<code>len(languages)</code>

Lists are useful when the order of items matters and the collection may need to change, such as a shopping cart, a list of tasks, or a set of course modules.

2. Tuples

A tuple is an ordered collection of values that cannot be changed after it is created. Tuples use parentheses `()`.

```
point = (10, 20)
print(point[0])
print(point[1])

student = ("Rahul", 21, "Python")
print(student)
```

Because tuples are immutable, you cannot append, remove, or replace individual values. This makes tuples useful for fixed groups of related values.

Feature	List	Tuple
Syntax	<code>[10, 20]</code>	<code>(10, 20)</code>
Ordered	Yes	Yes
Duplicates	Allowed	Allowed
Changeable	Yes	No

Example: a point such as **(10, 20)** has a fixed structure. A tuple can also be used to represent a fixed record such as **(name, age, city)**.

3. Dictionaries

A dictionary stores data as **key-value pairs**. Each key is used to find its corresponding value. Dictionaries use curly brackets `{ }`.

```
student = {
    "name": "Rahul",
    "age": 21,
    "course": "Python"
}

print(student["name"])
print(student["age"])
```

Keys should be unique. Values can be changed, added, or removed. A dictionary is especially useful when each piece of information has a meaningful label.

Common dictionary operations

Operation	Meaning	Example
dict[key]	Access a value	student["name"]
get()	Get a value safely	student.get("age")
update()	Add or update key-value pairs	student.update({"age": 22})
keys()	Get all keys	student.keys()
values()	Get all values	student.values()
items()	Get key-value pairs	student.items()
pop()	Remove a key and its value	student.pop("age")

Example: an AI application may store information about a user in a dictionary, such as name, language, preferences, and conversation settings.

4. Sets

A set is a collection of unique values. Sets do not keep duplicate values. Sets use curly brackets `{ }`.

```
skills = {"Python", "SQL", "Python", "Git"}
print(skills)

skills.add("Docker")
print(skills)
```

The repeated **Python** value appears only once because sets automatically keep unique values. Sets are useful when you mainly care about whether a value exists, rather than its position.

Common set operations

Operation	Meaning	Example
add()	Add an item	skills.add("Docker")
remove()	Remove an item	skills.remove("SQL")
len()	Count unique items	len(skills)
in	Check membership	"Python" in skills
union()	Combine values from two sets	a.union(b)
intersection()	Find common values	a.intersection(b)
difference()	Find values only in one set	a.difference(b)

Important: lists and tuples support indexing such as `items[0]`. Sets are not used for position-based access.

5. Choosing the Right Collection

Collection	Use when you need...	Example
List	An ordered, changeable group of items	Shopping cart
Tuple	An ordered, fixed group of items	Coordinates
Dictionary	Named information as key-value pairs	User profile
Set	Unique values and membership checks	Unique skills

6. Common Operations

Some operations work across multiple collection types. Learning these common patterns helps you move between different data structures.

Operation	Purpose	Example
len()	Count items	len(items)
in	Check whether a value exists	"Python" in items
for	Go through items one by one	for item in items:
append()	Add to a list	items.append("Python")
add()	Add to a set	skills.add("Python")
get()	Read a dictionary value safely	user.get("name")

Looping through a collection

A **for** loop can process each item in a collection one at a time.

```
languages = ["Python", "Java", "C++"]

for language in languages:
    print(language)
```

The same basic pattern can be used with tuples, sets, and dictionaries. For dictionaries, you can loop through keys, values, or key-value pairs.

```
student = {"name": "Rahul", "age": 21}

for key, value in student.items():
    print(key, value)
```

7. Real-World Use Cases

Collection	Real-world example	Typical data
List	Shopping cart	Product names or product IDs
List	Task manager	Pending tasks
Tuple	Location or coordinates	Latitude and longitude
Dictionary	User profile	Name, age, preferences
Dictionary	API response	Field names and values
Set	Unique visitors	User IDs
Set	Common skills	Skills shared by candidates

8. Quick Cheat Sheet

Concept	Remember
list []	Ordered and changeable collection. Duplicates are allowed.
tuple ()	Ordered collection that cannot be changed.
dict {key: value}	Stores data using key-value pairs.
set { }	Stores unique values. Duplicates are removed.
list[index]	Access a list or tuple item by position.
dict[key]	Access a dictionary value using its key.
append() / add()	Add an item to a list / set.
remove() / pop()	Remove items from collections.
keys() / values() / items()	Work with dictionary keys, values, or pairs.
union / intersection / difference	Common set operations.
len()	Count the number of items.
in	Check whether a value exists in a collection.

Key idea: Choose a collection based on how you need to store and use the data. Lists are flexible, tuples are fixed, dictionaries are labeled, and sets focus on uniqueness.